

Unordered Container

无序关联式容器

概述

自 C++11 标准起，四种基于 [哈希](#) 实现的无序关联式容器正式纳入了 C++ 的标准模板库中，分别是：unordered_set, unordered_multiset, unordered_map, unordered_multimap。

► 编译器不支持 C++11 的使用方法

它们与相应的关联式容器在功能，函数等方面有诸多共同点，而最大的不同点则体现在普通的关联式容器一般采用红黑树实现，内部元素按特定顺序进行排序；而这几种无序关联式容器则采用哈希方式存储元素，内部元素不以任何特定顺序进行排序，所以访问无序关联式容器中的元素时，访问顺序也没有任何保证。

采用哈希存储的特点使得无序关联式容器 **在平均情况下** 大多数操作（包括查找，插入，删除）都能在常数时间复杂度内完成，相较于关联式容器与容器大小成对数的时间复杂度更加优秀。

► Warning

由于无序关联式容器与相应的关联式容器在用途和操作中有很多共同点，这里不再介绍无序关联式容器的各种操作，这些内容读者可以参考 [关联式容器](#)。

制造哈希冲突

上文中提到了，在最坏情况下，对无序关联式容器进行一些操作的时间复杂度会与容器大小成线性关系。

在哈希函数确定的情况下，可以构造出数据使得容器内产生大量哈希冲突，导致复杂度达到上界。

在标准库实现里，每个元素的散列值是将值对一个质数取模得到的，更具体地说，是 [这个列表](#) 中的质数（g++ 6 及以前版本的编译器，这个质数一般是，g++ 7 及之后版本的编译器，这个质数一般是）。

因此可以通过向容器中插入这些模数的倍数来达到制造大量哈希冲突的目的。

自定义哈希函数

使用自定义哈希函数可以有效避免构造数据产生的大量哈希冲突。

要想使用自定义哈希函数，需要定义一个结构体，并在结构体中重载 () 运算符，像这样：

```
1 struct my_hash {  
2     size_t operator()(int x) const { return x; }  
3 };
```

当然，为了确保哈希函数不会被迅速破解（例如 Codeforces 中对使用无序关联式容器的提交进行 hack），可以试着在哈希函数中加入一些随机化函数（如时间）来增加破解的难度。

例如，在 [这篇博客](#) 中给出了如下哈希函数：

```

1 struct my_hash {
2     static uint64_t splitmix64(uint64_t x) {
3         x += 0x9e3779b97f4a7c15;
4         x = (x ^ (x >> 30)) * 0xbf58476d1ce4e5b9;
5         x = (x ^ (x >> 27)) * 0x94d049bb133111eb;
6         return x ^ (x >> 31);
7     }
8
9     size_t operator()(uint64_t x) const {
10         static const uint64_t FIXED_RANDOM =
11             chrono::steady_clock::now().time_since_epoch().count();
12         return splitmix64(x + FIXED_RANDOM);
13     }
14
15     // 针对 std::pair<LTint, int> 作为主键类型的哈希函数
16     size_t operator()(pair<uint64_t, uint64_t> x) const {
17         static const uint64_t FIXED_RANDOM =
18             chrono::steady_clock::now().time_since_epoch().count();
19         return splitmix64(x.first + FIXED_RANDOM) ^
20             (splitmix64(x.second + FIXED_RANDOM) >> 1);
21     }
22 };

```

写完自定义的哈希函数后，就可以通过 `unordered_map<LTint, int, my_hash> my_map;` 或者 `unordered_map<LTpair<LTint, int>, int, my_hash> my_pair_map;` 的定义方式将自定义的哈希函数传入容器了。

本页面最近更新：2023/12/16 23:06:11, [更新历史](#)

发现错误？想一起完善？ [在 GitHub 上编辑此页！](#)

本页面贡献者： [StudyingFather](#), [Enter-tainer](#), [Haohu Shen](#), [lr1d](#), [ouuan](#), [sbofgayschool](#), [Suyun514](#), [Tiphereth-A](#)

本页面的全部内容在 [CC BY-SA 4.0](#) 和 [SATA](#) 协议之条款下提供，附加条款亦可能应用